Федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский государственный технический университет имени Н.Э. Баумана»
(национальный исследовательский университет)
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ____ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ_____

КАФЕДРА _____КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)_____

О т ч е т

по домашнему заданию № 1

Название домашнего задания: Программирование с использованием библиотеки Qt

Дисциплина: Алгоритмизация и программирование

Студент гр. ИУ6-22Б _____ **М. Д. Акимов**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О. А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Цель работы: изучение средств создания графических интерфейсов в среде Qt Creator, освоение принципов событийного программирования и механизма сигналов-слотов, приобретение навыков реализации многооконных приложений, работа с динамическими структурами данных, а также практическое применение полиморфизма для создания анимированных графических объектов.

Задание:

1. Разработать приложение «Простейший калькулятор», выполняющее основные арифметические действия над вводимыми числами (согласно методическим указаниям).
2. Разработать приложение «Записная книжка», обеспечивающее добавление, поиск и удаление записей об абонентах (фамилия, имя, телефон, адрес).
3. Разработать программу, содержащую описание трёх графических объектов – отрезок, квадрат, правильный восьмиугольник. Используя механизм полиморфизма, привести объекты в одновременное вращение вокруг их геометрических центров с различными угловыми скоростями.

1. ЧАСТЬ 1. ПРОСТЕЙШИЙ КАЛЬКУЛЯТОР

1.1. Постановка задачи

Приложение должно имитировать работу настольного калькулятора с одной строкой ввода-вывода. На форме размещены: поле QLineEdit, кнопки операций «+», «-», «*», «/», кнопка «=», кнопка очистки «C» и кнопка «Выход». Пользователь вводит число, нажимает операцию, вводит следующее число и т.д. При нажатии «=» выводится результат. Повторное нажатие операции после результата использует его как первый операнд. Кнопка «C» сбрасывает состояние.

1.2. Диаграмма состояний интерфейса

Диаграмма описывает состояния интерфейса и переходы:



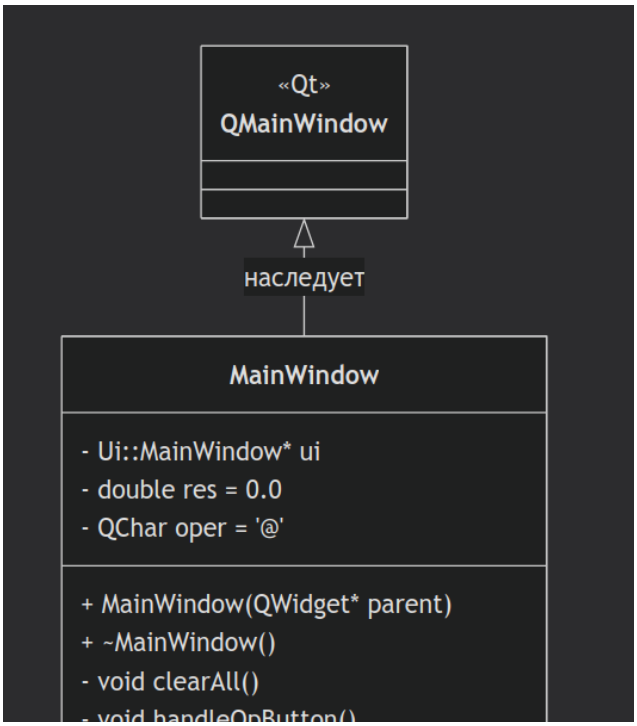
Рисунок 1 - Диаграмма состояний

- Состояние «Начало»: поле ввода содержит «0», фокус на поле.
- После ввода числа и нажатия кнопки операции – состояние «Ожидание второго операнда» (поле очищается).
- При нажатии «=» – состояние «Отображение результата» (результат выводится).
- Кнопка «C» всегда возвращает в состояние «Начало».
- Кнопка «Выход» завершает приложение.

1.3. Диаграмма классов

Класс MainWindow является наследником QMainWindow. Содержит:

- Ui::MainWindow *ui – автоматически сгенерированный интерфейс.
- double res – накопленный результат.
- QChar oper – последняя нажатая операция.
- Методы: конструктор, деструктор, clearAll(), handleOpButton().

- 

Взаимодейств C, pushButton_plus, pushButton_m Button_exit.

Ниже предст граммы:

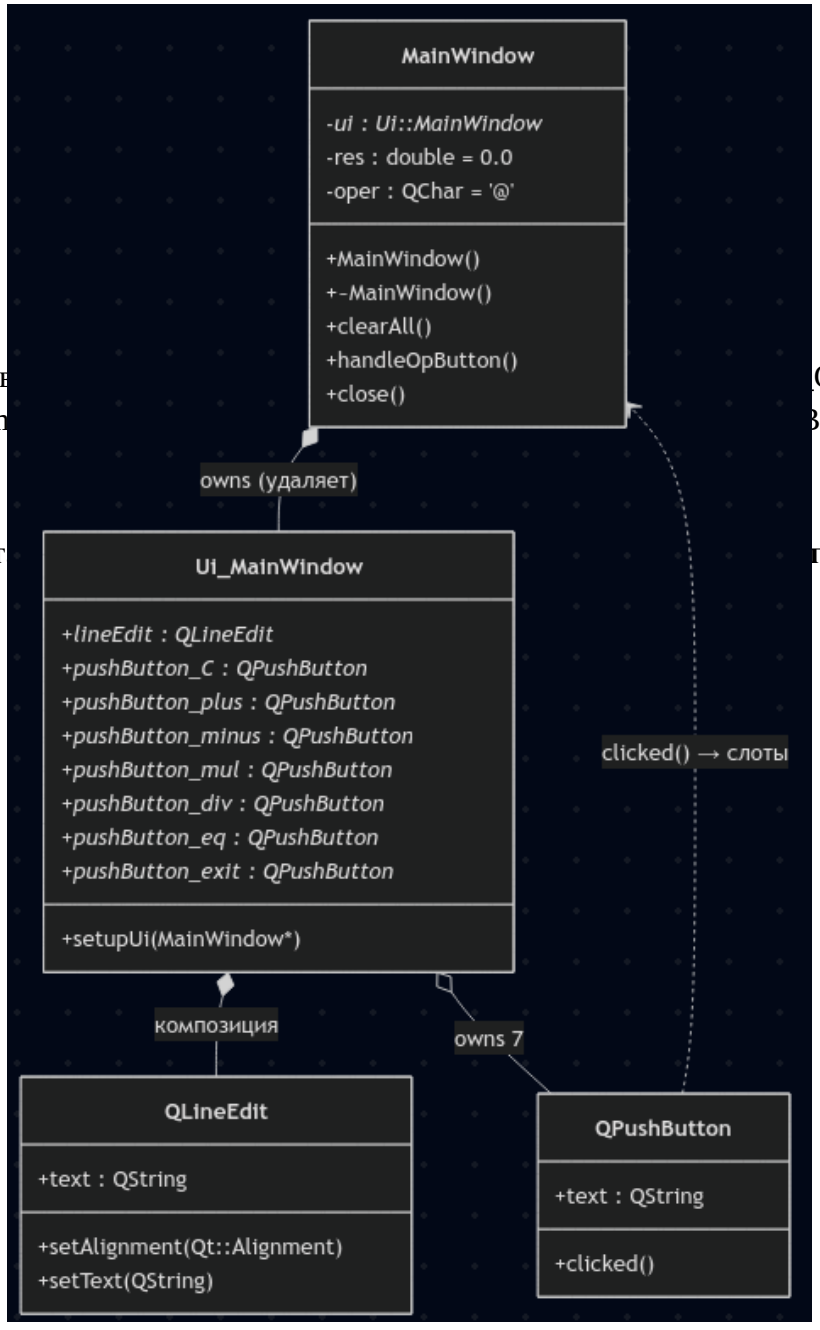


Рисунок 3 - Объектная декомпозиция

1.5. Текст программы (основные файлы)

Main.cpp

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
   QApplication a(argc, argv);
   MainWindow w;
   w.show();
   return a.exec();
}
```

mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QMainWindow>

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
   Q_OBJECT
public:
   explicit MainWindow(QWidget *parent = nullptr);
   ~MainWindow();

private:
   Ui::MainWindow *ui;
   double res = 0.0;
   QChar  oper = '@';
   void clearAll();
   void handleOpButton();
};
#endif // MAINWINDOW_H
```

mainwindow.cpp

```
#include "mainwindow.h"
#include "./ui_mainwindow.h"
#include <QPushButton>

MainWindow::MainWindow(QWidget *parent)
   : QMainWindow(parent), ui(new Ui::MainWindow)
{
   ui->setupUi(this);
   ui->lineEdit->setFocus();
   connect(ui->pushButton_C, &QPushButton::clicked, this, &MainWindow::clearAll);
   connect(ui->pushButton_plus,  &QPushButton::clicked, this, &MainWindow::handleOpButton);
   connect(ui->pushButton_minus, &QPushButton::clicked, this, &MainWindow::handleOpButton);
   connect(ui->pushButton_mul,   &QPushButton::clicked, this, &MainWindow::handleOpButton);
   connect(ui->pushButton_div,   &QPushButton::clicked, this, &MainWindow::handleOpButton);
   connect(ui->pushButton_eq,    &QPushButton::clicked, this, &MainWindow::handleOpButton);
```

```
    connect(ui->pushButton_exit,  &QPushButton::clicked, this, &QWidget::close);
}

MainWindow::~MainWindow() { delete ui; }

void MainWindow::clearAll()
{
    res = 0.0; oper = '@';
    ui->lineEdit->clear();
    ui->lineEdit->setFocus();
}

void MainWindow::handleOpButton()
{
    auto *btn = qobject_cast<QPushButton*>(sender());
    if (!btn) return;
    double x = ui->lineEdit->text().toDouble();
    if (oper == '+')      res += x;
    else if (oper == '-') res -= x;
    else if (oper == '*') res *= x;
    else if (oper == '/') res /= x;
    else                  res  = x;
    oper = btn->text().at(0);
    if (oper == '=')
        ui->lineEdit->setText(QString::number(res));
    else
        ui->lineEdit->clear();
    ui->lineEdit->setFocus();
}
```

1.6. Тестирование

Тест 1. Простое вычисление

Ввод: 15, «+», 27, «=»

Ожидаемый результат: 42.

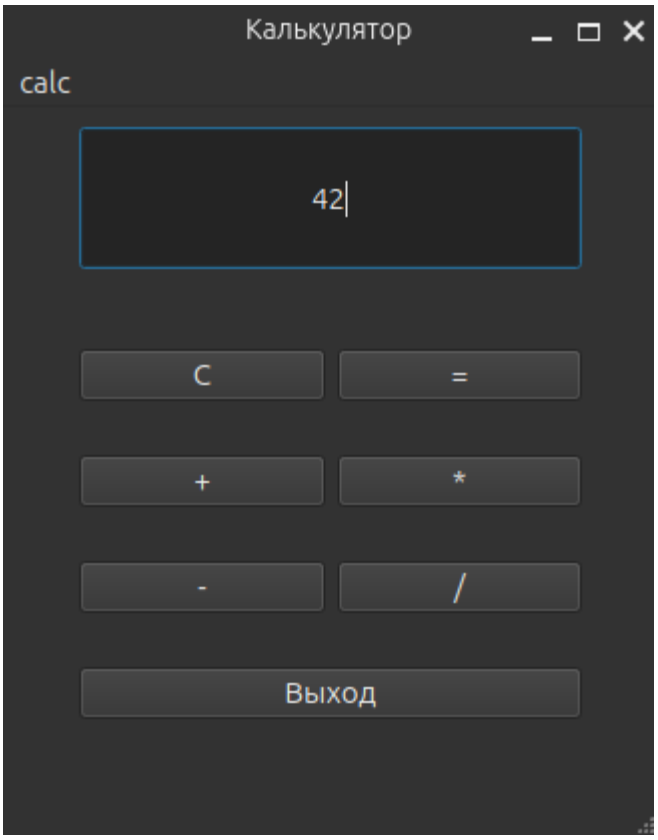


Рисунок 4 - Результат работы программы (Тест 1)

Тест 2. Цепочка операций

Ввод: 10, «-», 3, «*», 4, «=»

Порядок выполнения: (10 – 3) = 7, затем 7 * 4 = 28.

Ожидаемый результат: 28.

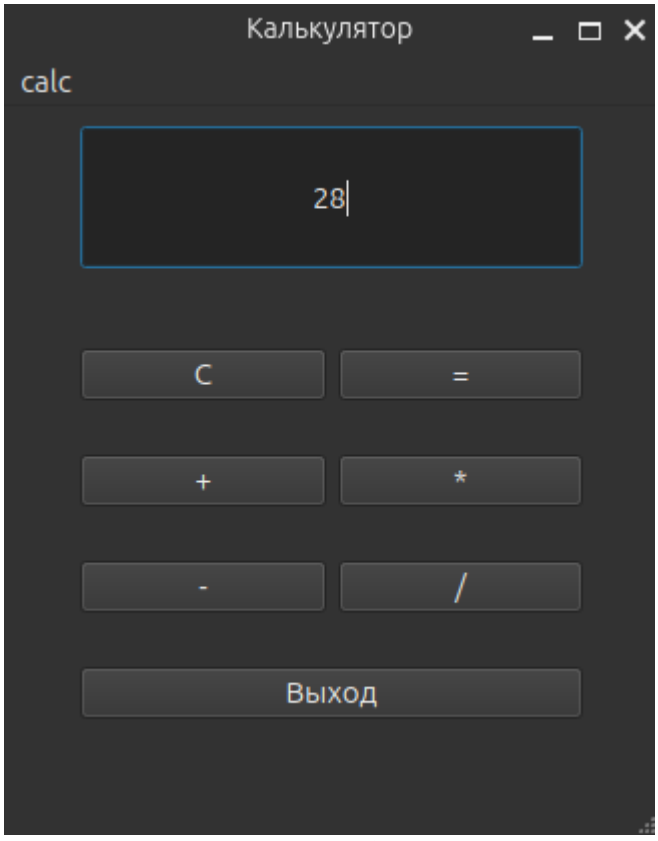


Рисунок 5 - Результат работы программы (Тест 2)

2. ЧАСТЬ 1. ЗАПИСНАЯ КНИЖКА

2.1. Постановка задачи

Приложение должно обеспечивать:

- добавление контакта (фамилия, имя, телефон, адрес);
- поиск контакта по любому сочетанию полей (допустимо частичное совпадение – в реализации реализовано точное совпадение либо пустое поле игнорируется);
- удаление найденного контакта;
- многооконный интерфейс: главное окно с кнопками «Добавление записей» и «Поиск записей», открывающими соответствующие формы.

2.2. Реализованные формы интерфейса

- **Главное окно**: три кнопки – «Добавление записей», «Поиск записей», «Выход».
- **Форма добавления**: четыре поля ввода (фамилия, имя, телефон, адрес) и две кнопки – «Добавить», «Конец».
- **Форма поиска**: те же четыре поля, кнопки «Найти», «Удалить», «Конец». Поиск последовательно перебирает найденные записи, запрашивая подтверждение через

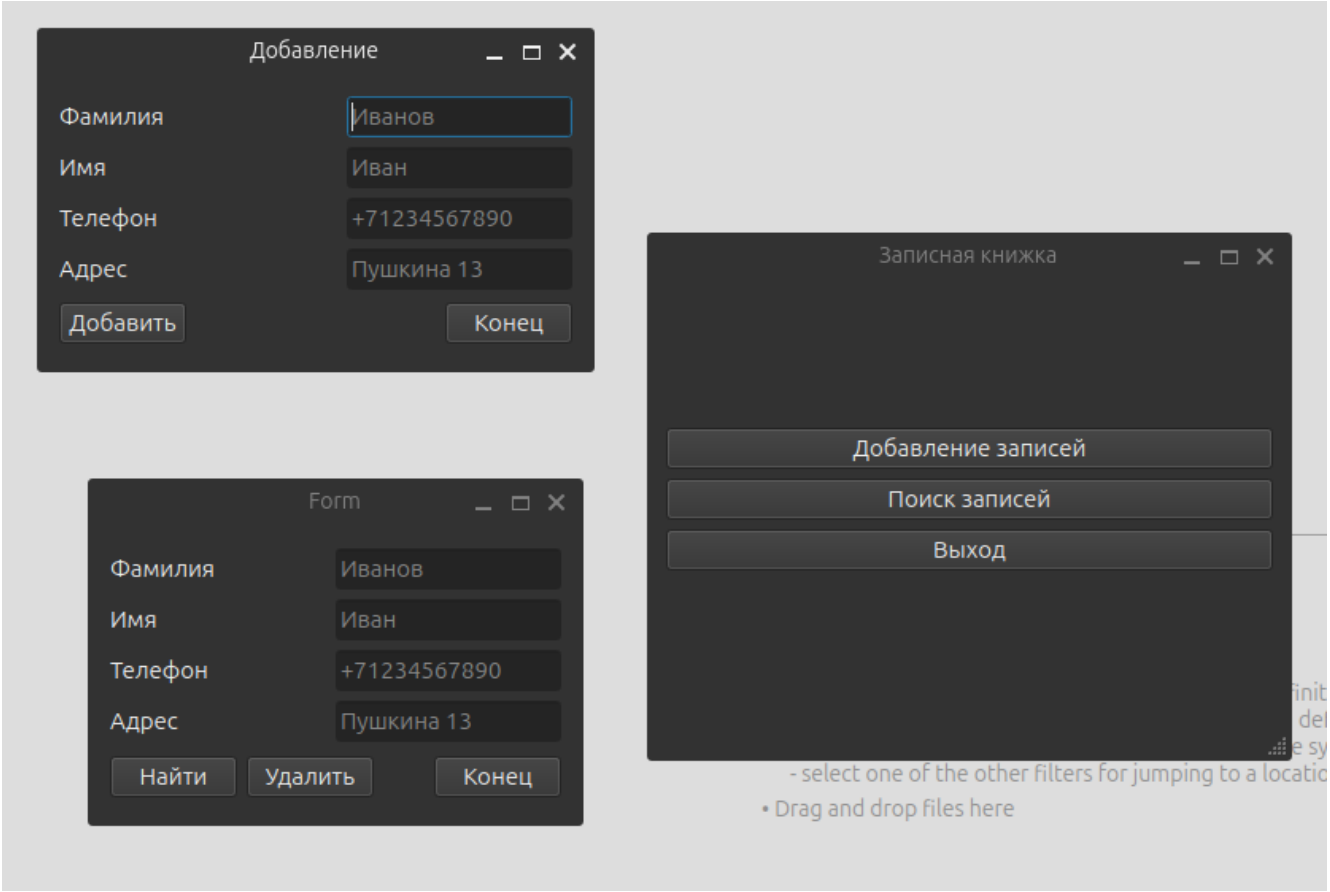


Рисунок 6 - Вид всех форм приложения

диалог.

2.3. Диаграммы состояний интерфейса

Главное окно → при нажатии «Добавление записей» открывается форма AddForm. При нажатии «Поиск записей» открывается FindForm. Выход из главного окна завершает приложение.



Рисунок 7 - Диаграмма состояний главного окна

Форма добавления: ввод данных → кнопка «Добавить» вызывает ContactList::addContact(),



Рисунок 8 - Диаграмма состояний формы добавления

очищает поля, окно остается открытым. Кнопка «Конец» закрывает форму.

Форма поиска: после ввода критериев кнопка «Найти» запускает поиск. Если контакт найден, выводится диалог «Пользователь, которого вы искали? … (Yes/No)». При Yes – поиск завершается (контакт признан искомым). При No – продолжается поиск следующего совпадения. Кнопка «Удалить» аналогично ищет и при подтверждении удаляет запись. Если контактов больше не найдено, выводится сообщение.

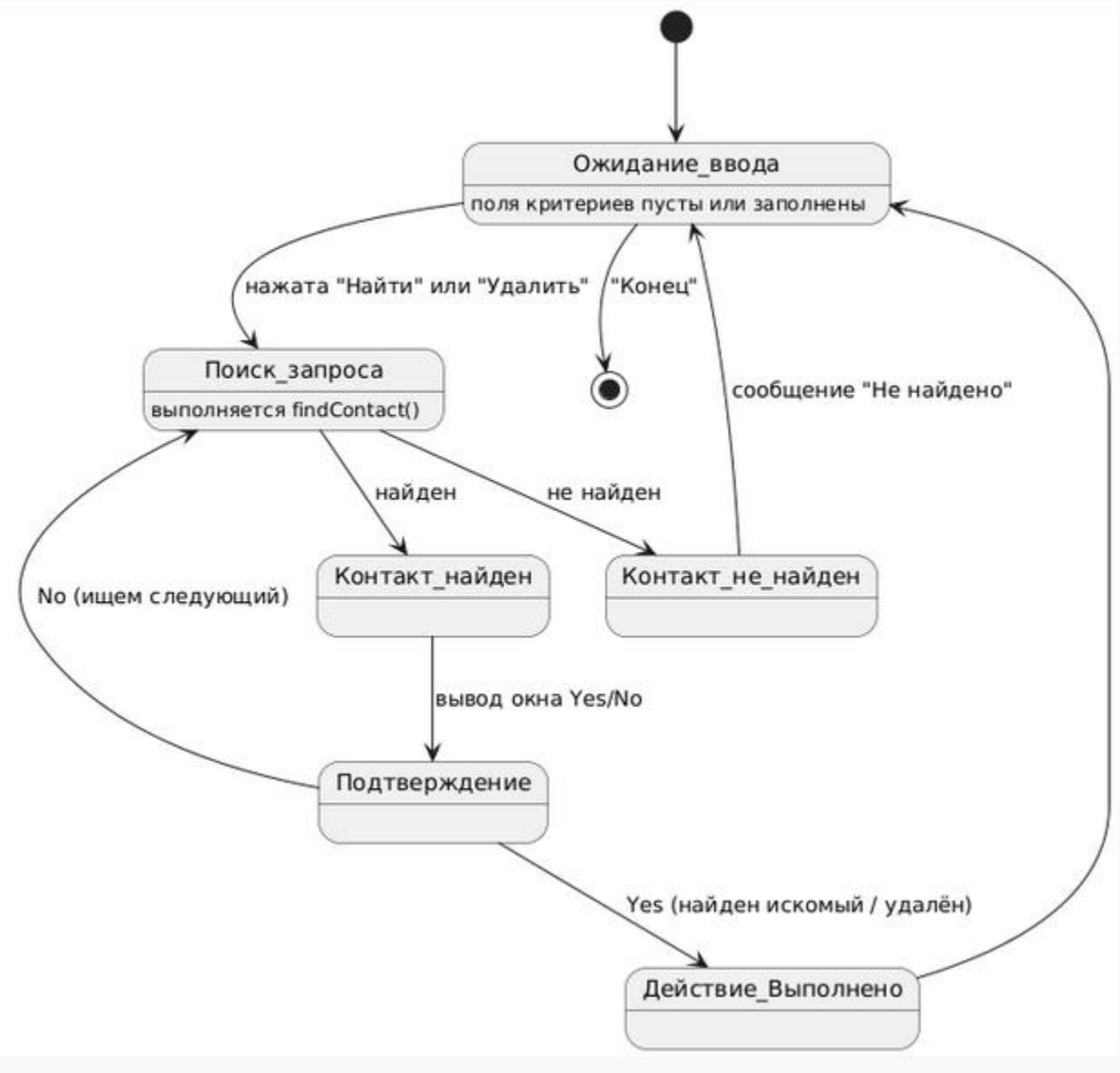


Рисунок 9 - Диаграмма состояний формы поиска

2.4. Диаграммы классов интерфейса

Классы данных:

- Contact – хранит поля QString name, surname, phone, address и указатель next.
- ContactList – односвязный список, содержит head, методы addContact, deleteContact, findContact, деструктор. Не наследует QObject.

Классы форм:

- MainWindow (QMainWindow) – содержит объект ContactList mylist, объекты AddForm add и FindForm find. Слоты открывают формы (показывают как независимые окна).
- AddForm (QWidget) – принимает указатель на ContactList. Слот on_pushButton_Add_clicked() добавляет запись.
- FindForm (QWidget) – принимает указатель на ContactList. Поля: Contact* curr. Слоты поиска и удаления.

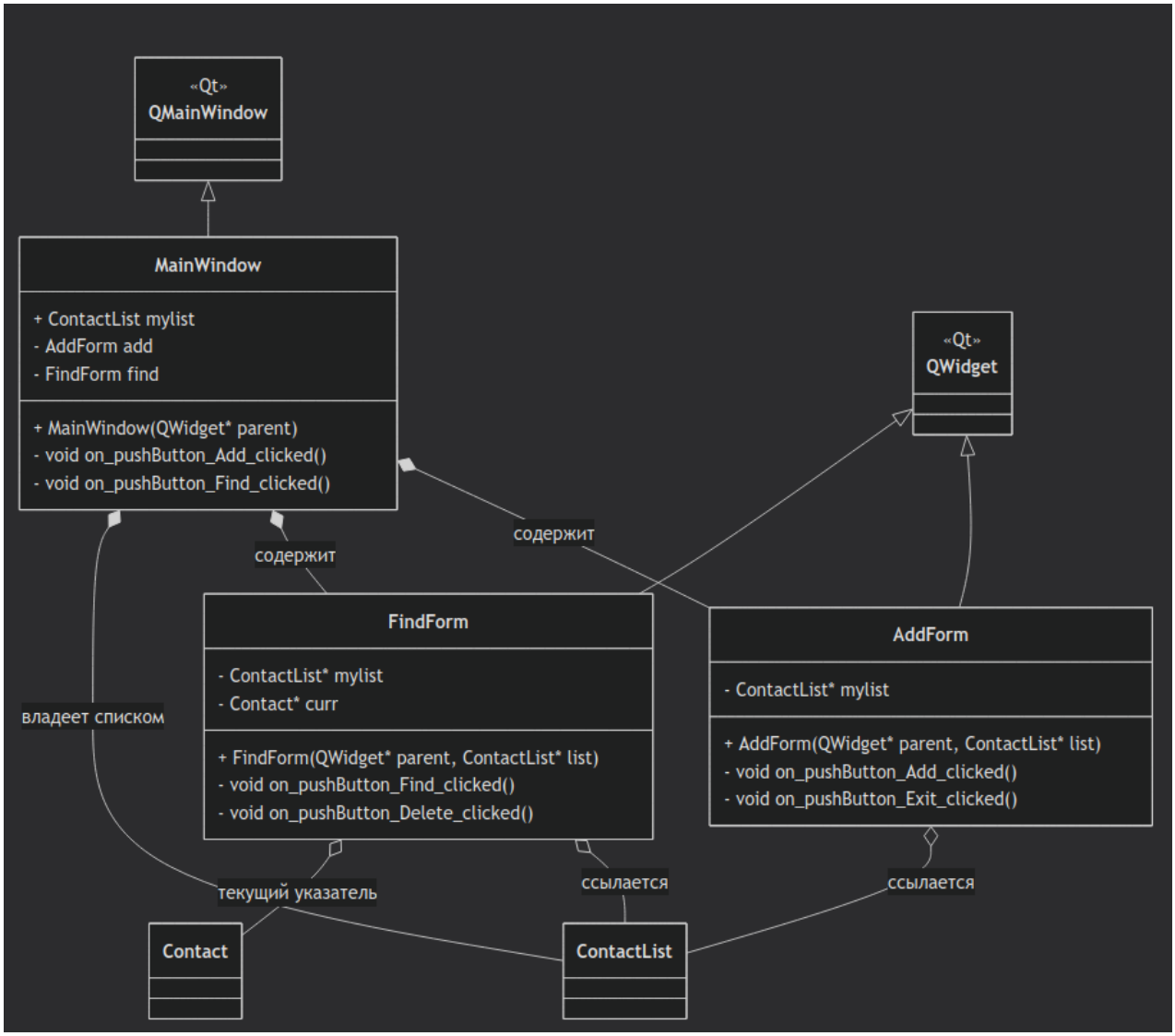


Рисунок 10 - Диаграмма классов интерфейса

Все формы визуализируются методом show().

2.5. Текст программы (ключевые фрагменты)

contact.h

```
#ifndef CONTACT_H
#define CONTACT_H
#include <QString>
class Contact {
    QString name, surname, phone, address;
public:
    Contact* next;
    Contact() {}
    Contact(QString n, QString s, QString p, QString a)
        : name(n), surname(s), phone(p), address(a), next(nullptr) {}
    QString getName() const { return name; }
    QString getSurname() const { return surname; }
    QString getPhone() const { return phone; }
    QString getAddress() const { return address; }
};
#endif
```

contactlist.h / contactlist.cpp

```
// contactlist.h
#ifndef CONTACT_LIST_H
#define CONTACT_LIST_H
#include "contact.h"
class ContactList {
public:
    Contact *head;
    ContactList() : head(nullptr) {}
    Contact* addContact(QString name, QString surname, QString phone, QString address);
    void deleteContact(Contact* contact);
    Contact* findContact(Contact* start, const QString name, const QString surname,
                         const QString phone, const QString address) const;
    ~ContactList();
};
#endif

// contactlist.cpp (методы)
Contact* ContactList::addContact(...) {
    Contact* newContact = new Contact(name, surname, phone, address);
    if (!head) head = newContact;
    else {
        Contact* temp = head;
        while (temp->next) temp = temp->next;
        temp->next = newContact;
    }
    return newContact;
}
void ContactList::deleteContact(Contact* contact) {
    Contact *temp = head, *prev = nullptr;
    while (temp != contact) { prev = temp; temp = temp->next; }
    if (prev) prev->next = temp->next;
    else head = temp->next;
    delete temp;
}
Contact* ContactList::findContact(Contact* start, const QString name,
    const QString surname, const QString phone, const QString address) const {
    Contact* temp = start;
    while (temp) {
        if ((temp->getName() == name || name.isEmpty()) &&
            (temp->getSurname() == surname || surname.isEmpty()) &&
            (temp->getPhone() == phone || phone.isEmpty()) &&
            (temp->getAddress() == address || address.isEmpty()))
            return temp;
        temp = temp->next;
    }
    return nullptr;
}
ContactList::~ContactList() {
    while (head) {
        Contact* toDelete = head;
        head = head->next;
        delete toDelete;
    }
}
```

addform.h / addform.cpp

```
#ifndef ADDFORM_H

#define ADDFORM_H



#include <QWidget>

#include "contactList.h"
```

```
namespace Ui {
class AddForm;
}

class AddForm : public QWidget
{
    Q_OBJECT

public:
    explicit AddForm(QWidget *parent = nullptr, ContactList* list = nullptr);
    ~AddForm();

private slots:
    void on_pushButton_Exit_clicked();
    void on_pushButton_Add_clicked();

private:
    Ui::AddForm *ui;
    ContactList* mylist;
};

#endif // ADDFORM_H
#include "addform.h"
#include "ui_addform.h"

AddForm::AddForm(QWidget *parent, ContactList *list)
    : QWidget(parent)
    , ui(new Ui::AddForm)
    , mylist(list)
{
    ui->setupUi(this);
}

AddForm::~AddForm()
{
    delete ui;
```

```
}


void AddForm::on_pushButton_Exit_clicked()
{
    this->close();
}


void AddForm::on_pushButton_Add_clicked()
{
    QString name = ui->lineEdit_Name->text();
    QString surname = ui->lineEdit_Surname->text();
    QString phone = ui->lineEdit_Phone->text();
    QString address = ui->lineEdit_Address->text();


    mylist->addContact(name, surname, phone, address);


    ui->lineEdit_Name->clear();
    ui->lineEdit_Surname->clear();
    ui->lineEdit_Phone->clear();
    ui->lineEdit_Address->clear();
}
```

mainwindow.h / mainwindow.cpp

```
// mainwindow.h
class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
    ContactList mylist;
private slots:
    void on_pushButton_Add_clicked();
    void on_pushButton_Find_clicked();
private:
    Ui::MainWindow *ui;
    AddForm add;
    FindForm find;
};
// mainwindow.cpp
MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent), ui(new Ui::MainWindow) {
    ui->setupUi(this);
}
void MainWindow::on_pushButton_Add_clicked() {
    add.show();
}
void MainWindow::on_pushButton_Find_clicked() {
    find.show();
}
```

2.6. Тестирование и результаты

Тест 1. Добавление контакта. Ввели фамилию «Иванов», имя «Петр», телефон «+79991234567», адрес «ул. Мира, 5». Контакт добавлен, поля очищены.

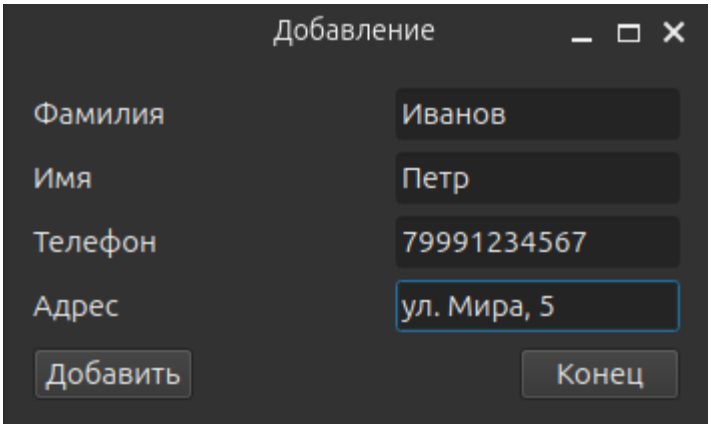


Рисунок 11 - Добавление контакта

Тест 2. Поиск по фамилии. В форме поиска указали фамилию «Иванов», остальные поля пустые. Нажали

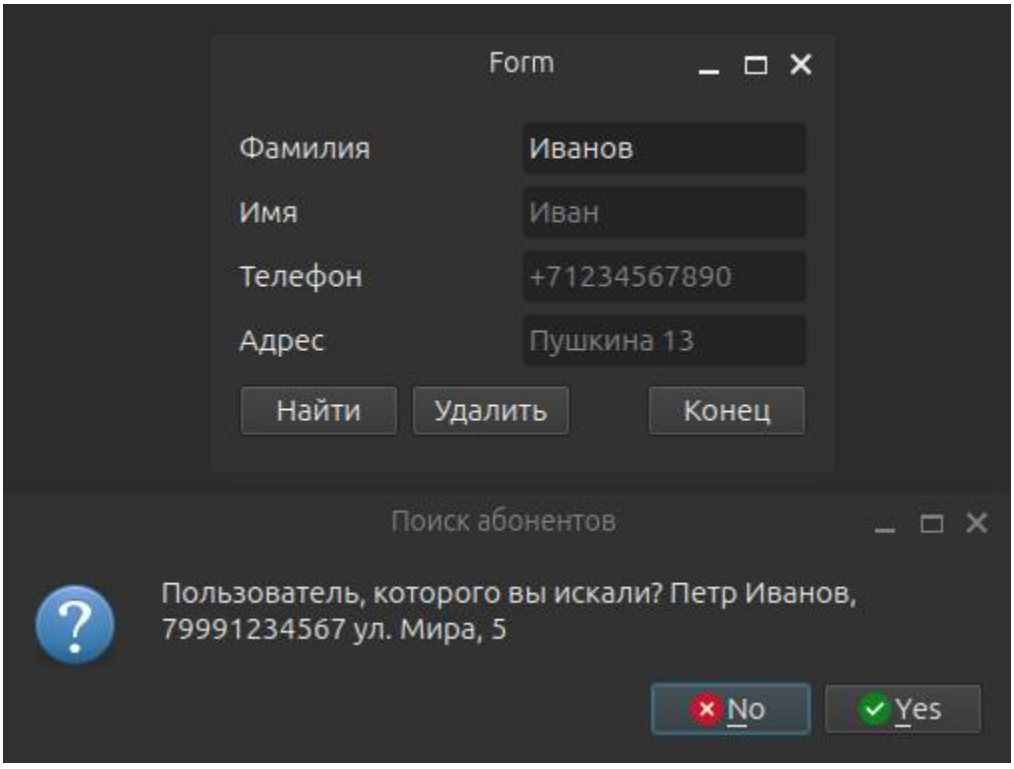


Рисунок 12 - Поиск контакта по фамилии

«Найти». Появился диалог с найденным контактом, ответили Yes. Поиск успешен.

ё1

Тест 3. Поиск с несколькими совпадениями. Добавили двух «Петровых». При поиске по фамилии «Петров» первый раз найден первый, после No – найден второй. При Yes поиск остановлен.

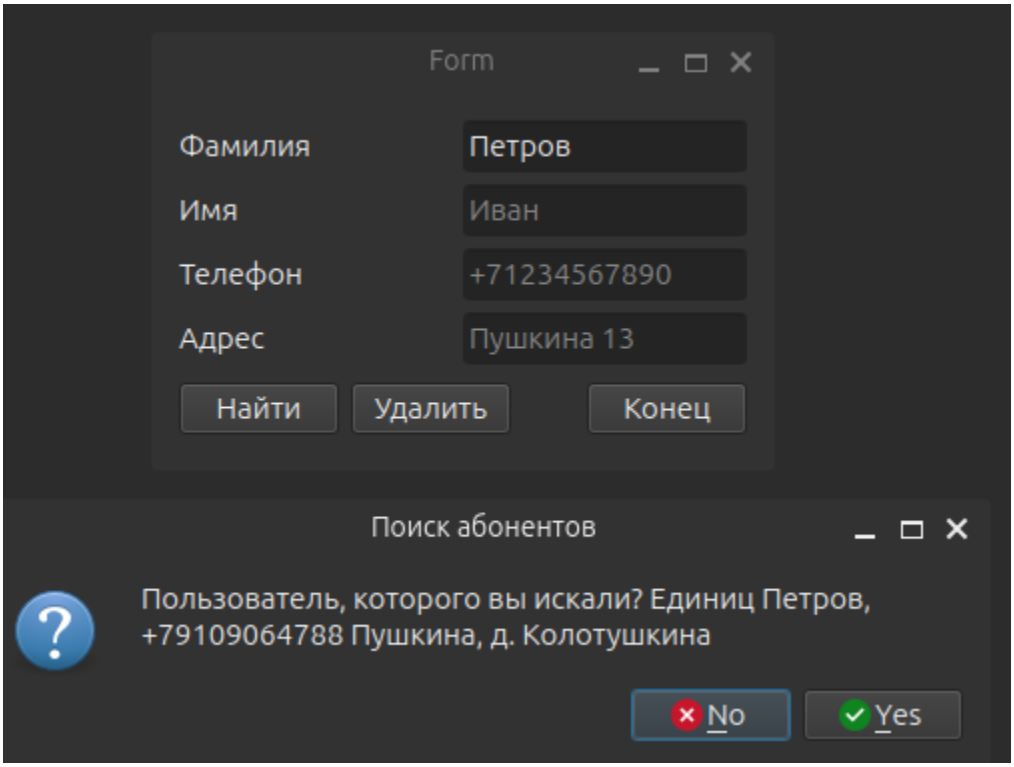


Рисунок 13 - Поиск с несколькими совпадениями (1)

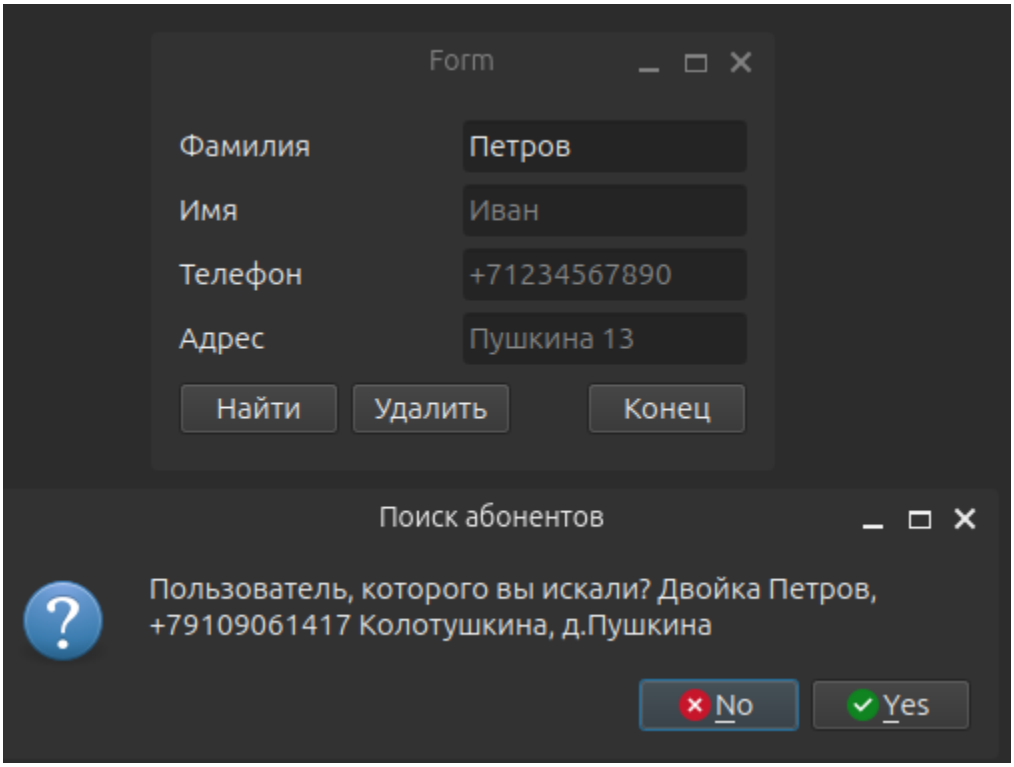


Рисунок 14 - Поиск с несколькими совпадениями (2)

Тест 4. Удаление. Нашли того же «Иванова», нажали «Удалить», подтвердили. При повторном поиске контакт не найден.

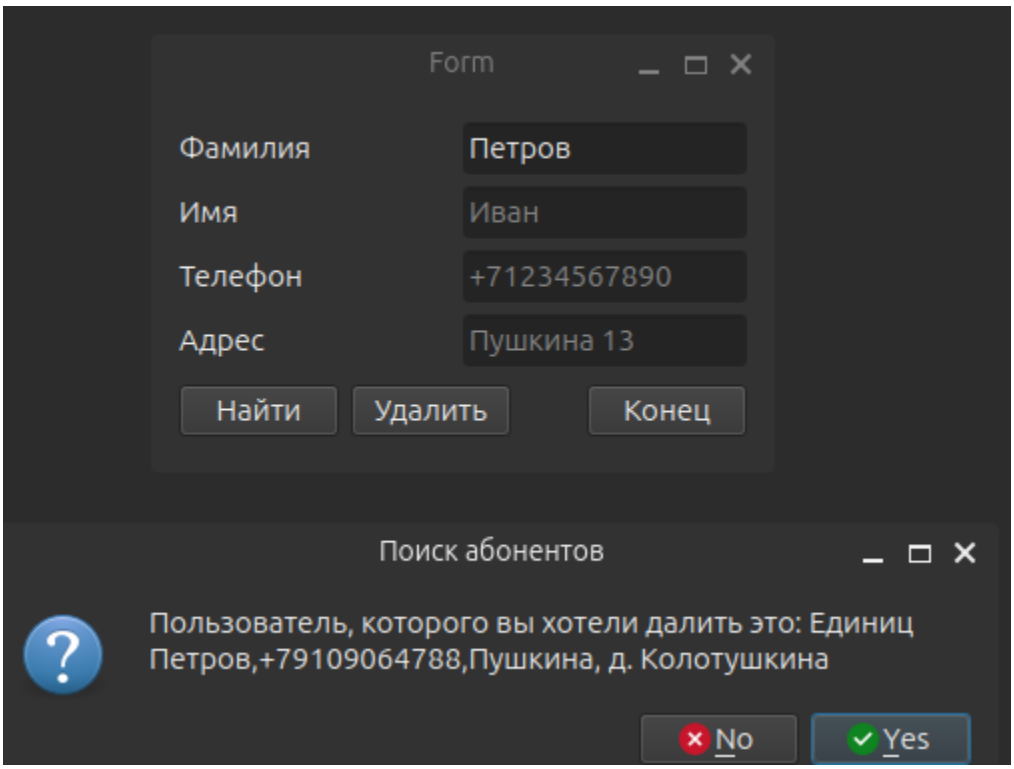


Рисунок 15 - Удаление пользователя(1)

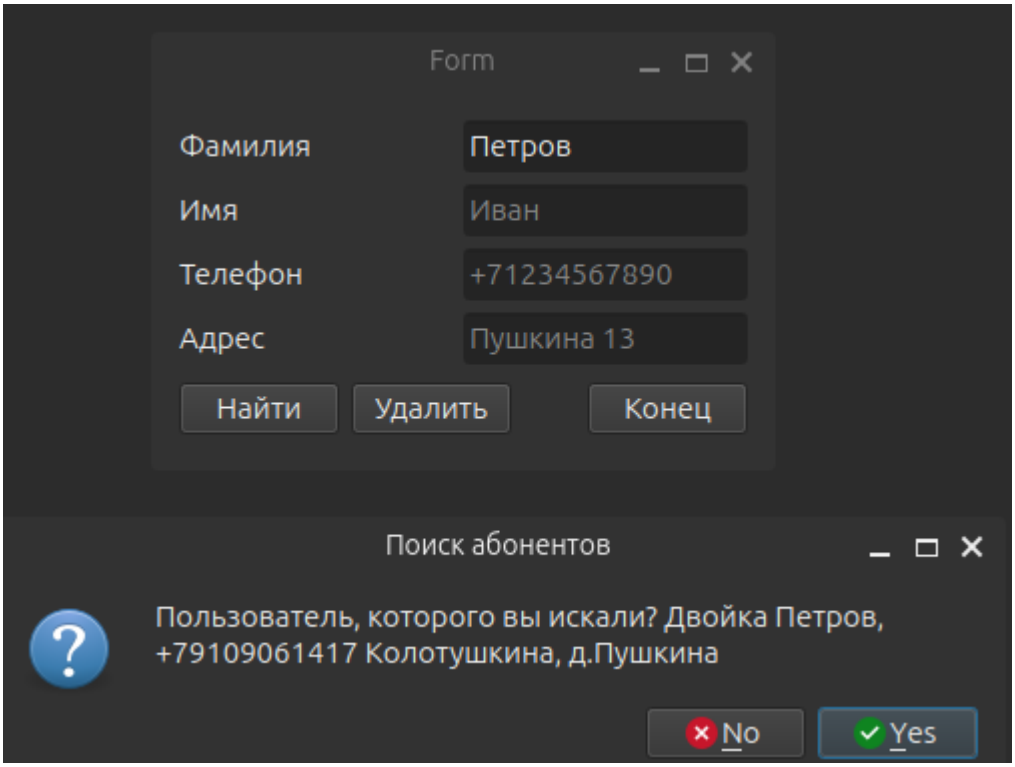


Рисунок 16 - Удаление пользователя (2)

3. ЧАСТЬ 2. РЕАЛИЗАЦИЯ ДВИЖЕНИЯ С ИСПОЛЬЗОВАНИЕМ СЛОЖНОГО ПОЛИМОРФИЗМА

3.1. Постановка задачи

Создать программу, в которой описаны три графических объекта: отрезок (MyLine), квадрат (MyRect) и правильный восьмиугольник (MyOctagon). Все объекты наследуют абстрактный базовый класс Figur a, содержащий координаты центра, половинный размер, угол поворота и скорость, а также чисто виртуальный метод draw(). Приложение должно запускать таймер, обновляющий угол поворота каждого объекта на величину, пропорциональную его скорости, и перерисовывающий виджет. Таким образом, объекты вращаются одновременно с различными угловыми скоростями.

3.2. Объектная декомпозиция и диаграмма классов

Классы предметной области (фигуры):

- Figura (абстрактный) – поля: int x, y, halflen, float angle, speed; виртуальный метод draw(QPainter*) = 0; метод move(float deltaTime), увеличивающий angle на speed * deltaTime.
- MyLine : public Figura – реализует draw, рисуя отрезок из центра с поворотом.
- MyRect : public Figura – рисует квадрат, центр в (x,y), поворот.
- MyOctagon : public Figura – рисует правильный восьмиугольник (вершины вычисляются по полярным координатам).

Классы интерфейса:

- Window (QWidget) – содержит объект Area (область отрисовки) и кнопку «Завершить». Компоновка вертикальная.
- Area (QWidget) – владеет объектами фигур (myline, myrect, myoctagon). Запускает таймер с интервалом 50 мс. В paintEvent вызывает move(1.0f) для каждой фигуры и затем draw(). В timerEvent вызывает update().

Диаграмма классов:

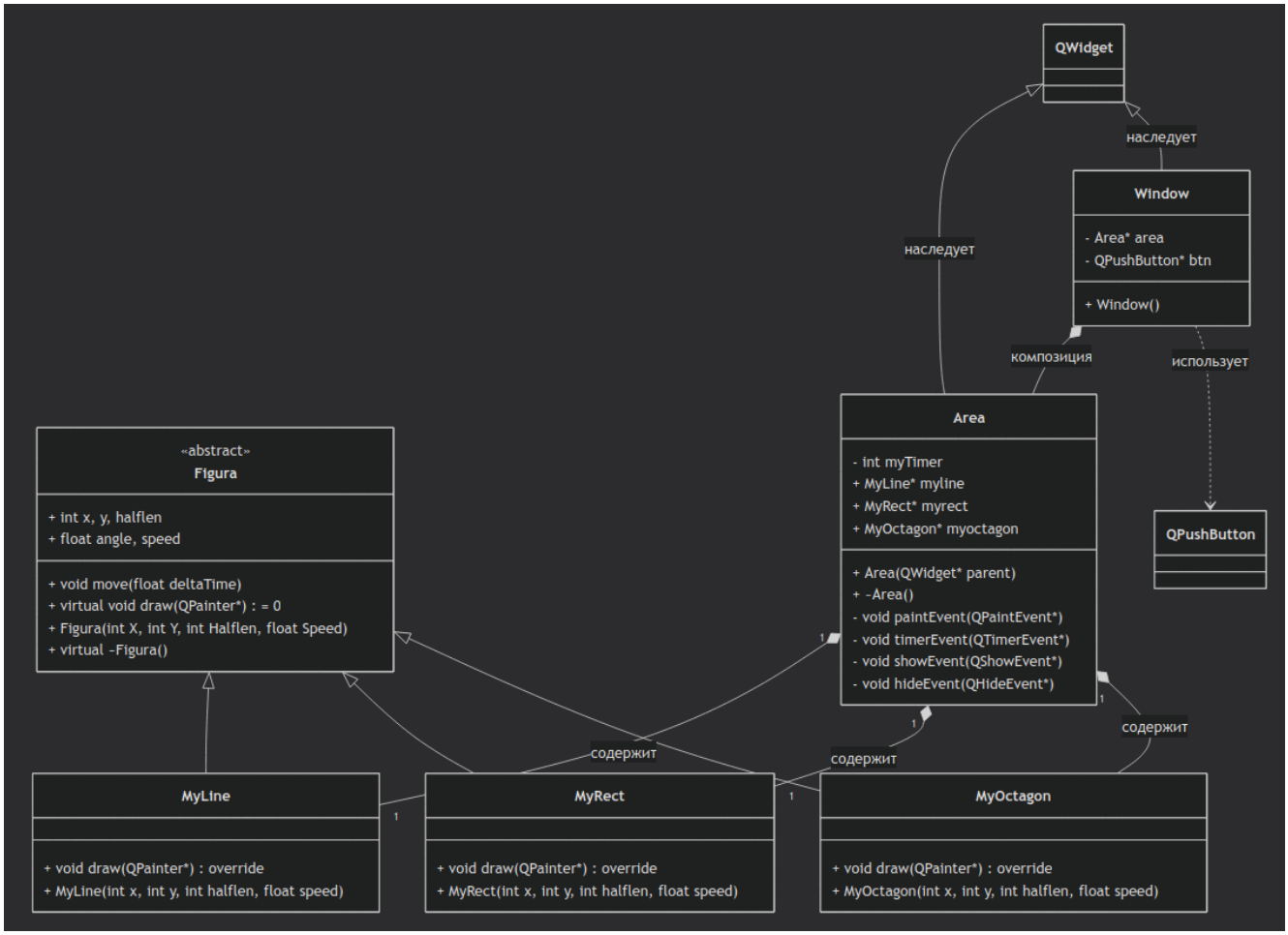


Рисунок 17 - Диаграмма классов

3.3. Граф состояний пользовательского интерфейса

Приложение имеет единственное окно. При запуске создаётся объект Window, который отображает область Area и кнопку. Таймер сразу запускается в showEvent. Состояния:

- **Ожидание**: окно видимо, фигуры вращаются (перерисовка каждые 50 мс).
- При нажатии кнопки «Завершить» окно закрывается.
  Граф представляет собой простой цикл: отображение – обновление кадров – закрытие.

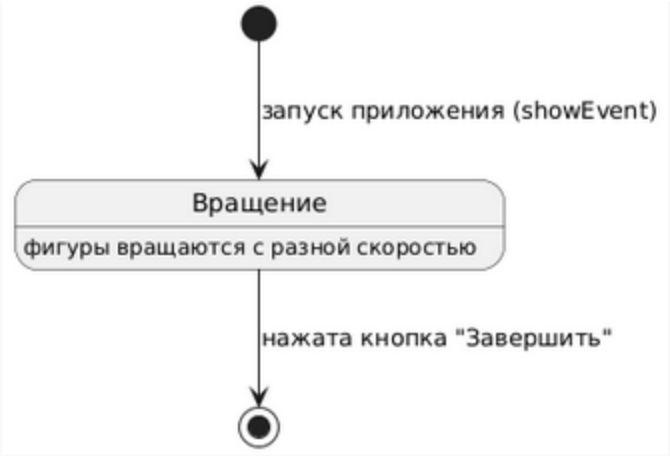


Рисунок 18 - Граф состояний пользовательского интерфейса

3.4. Текст программы

figura.h

```
#ifndef FIGURA_H
#define FIGURA_H
#include <QPainter>
#include <cmath>

class Figura {
public:
    int x, y, halflen;
    float angle, speed;
    virtual void draw(QPainter *Painter) = 0;
    Figura(int X, int Y, int Halflen, float Speed)
        : x(X), y(Y), halflen(Halflen), angle(0.0f), speed(Speed) {}
    void move(float deltaTime = 1.0f) { angle += speed * deltaTime; }
    virtual ~Figura() {}
};

class MyLine : public Figura {
public:
    void draw(QPainter *Painter) override;
    MyLine(int x, int y, int halflen, float speed) : Figura(x,y,halflen,speed) {}
```

```
};

class MyRect : public Figura {
public:
    void draw(QPainter *Painter) override;
    MyRect(int x, int y, int halflen, float speed) : Figura(x,y,halflen,speed) {}
};

class MyOctagon : public Figura {
public:
    void draw(QPainter *Painter) override;
    MyOctagon(int x, int y, int halflen, float speed) : Figura(x,y,halflen,speed) {}
};
#endif
```

figura.cpp

```
#include "figura.h"
#include <QTransform>
#include <QPolygonF>

void MyLine::draw(QPainter *Painter) {
    QTransform t;
    t.translate(x,y).rotateRadians(angle).translate(-x,-y);
    Painter->save();
    Painter->setTransform(t, true);
    Painter->drawLine(x - halflen, y, x + halflen, y);
    Painter->restore();
}
void MyRect::draw(QPainter *Painter) {
    QTransform t;
    t.translate(x,y).rotateRadians(angle).translate(-x,-y);
    Painter->save();
    Painter->setTransform(t, true);
    QRect rect(x - halflen, y - halflen, 2*halflen, 2*halflen);
```

```
    Painter->drawRect(rect);
    Painter->restore();
}
void MyOctagon::draw(QPainter *Painter) {
    const int n = 8;
    double step = 2 * M_PI / n;
    QPolygonF poly;
    for (int i = 0; i < n; ++i)
        poly << QPointF(halflen * cos(i*step), halflen * sin(i*step));
    QTransform t;
    t.translate(x,y).rotateRadians(angle);
    Painter->save();
    Painter->setTransform(t, true);
    Painter->drawPolygon(poly);
    Painter->restore();
}
```

area.h / area.cpp

```
// area.h
class Area : public QWidget {
    int myTimer;
public:
    Area(QWidget *parent = nullptr);
    ~Area();
    MyLine *myline;
    MyRect *myrect;
    MyOctagon *myoctagon;
protected:
    void paintEvent(QPaintEvent *) override;
    void timerEvent(QTimerEvent *) override;
    void showEvent(QShowEvent *) override;
    void hideEvent(QHideEvent *) override;
};
```

```
// area.cpp (конструктор)
Area::Area(QWidget *parent) : QWidget(parent) {
    setFixedSize(300,200);
    myline = new MyLine(80,100,50,0.02f);
    myrect = new MyRect(220,100,50,-0.01f);
    myoctagon = new MyOctagon(150,170,30,0.015f);
}
```

window.h / window.cpp

```
class Window : public QWidget {
    Area *area;
    QPushButton *btn;
public:
    Window() {
        setWindowTitle("Обработка событий");
        area = new Area(this);
        btn = new QPushButton("Завершить", this);
        auto *layout = new QVBoxLayout(this);
        layout->addWidget(area);
        layout->addWidget(btn);
        connect(btn, SIGNAL(clicked()), this, SLOT(close()));
    }
};
```

3.5. Тестирование

При запуске приложения отображается окно размером 300×200. В нём непрерывно вращаются:

- красный отрезок в точке (80,100) длиной 100 пикселей с небольшой скоростью;
- зелёный квадрат в (220,100) со стороной 100 пикселей, вращающийся в противоположную сторону;
- синий правильный восьмиугольник в (150,170) радиусом 30 пикселей.
  Все фигуры вращаются плавно, визуально заметна разница скоростей. Нажатие кнопки «Завершить» закрывает приложение. Таймер корректно останавливается при скрытии окна.

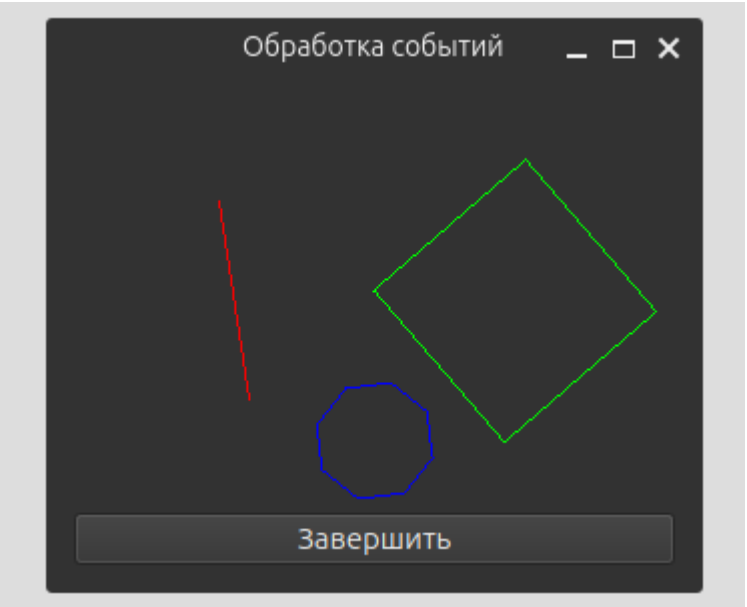


Рисунок 19 - Результат работы программы

ВЫВОДЫ

В результате выполнения домашнего задания были освоены принципы создания графических интерфейсов с помощью Qt Creator. Реализован простейший калькулятор, демонстрирующий работу с сигналами и слотами, использование компоновщиков и событийного программирования. Разработано многооконное приложение «Записная книжка», которое иллюстрирует организацию диалоговых окон, работу с пользовательскими данными и динамическими структурами (односвязный список). В третьей части применён полиморфизм для построения иерархии графических фигур и реализована анимация вращения. Все программы протестированы, их работоспособность подтверждена.

Полученные навыки могут быть использованы для создания более сложных оконных приложений на языке C++ с использованием библиотеки Qt.